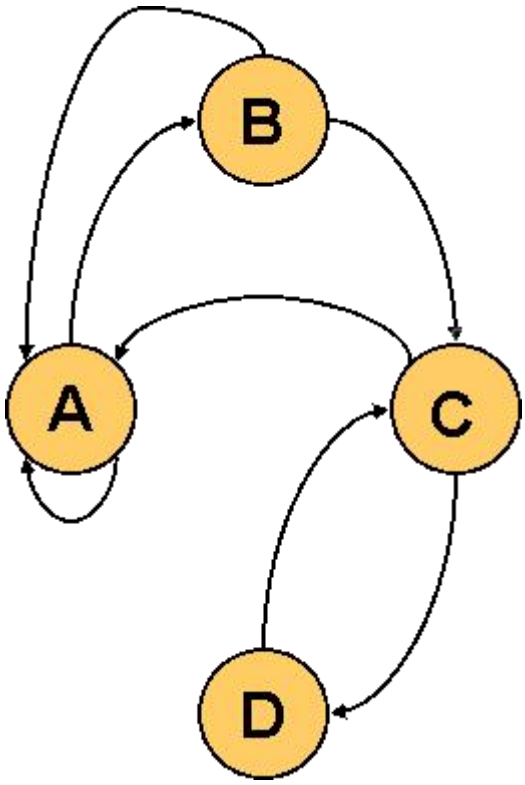
- “节点”，表示状态
- “弧线”，连接节点并表示转换规则。

在 WSF 中，“状态”可以表示几乎任何东西：

- 等待某物进入范围。
- 等待传感器获取。
- 等待武器引爆。

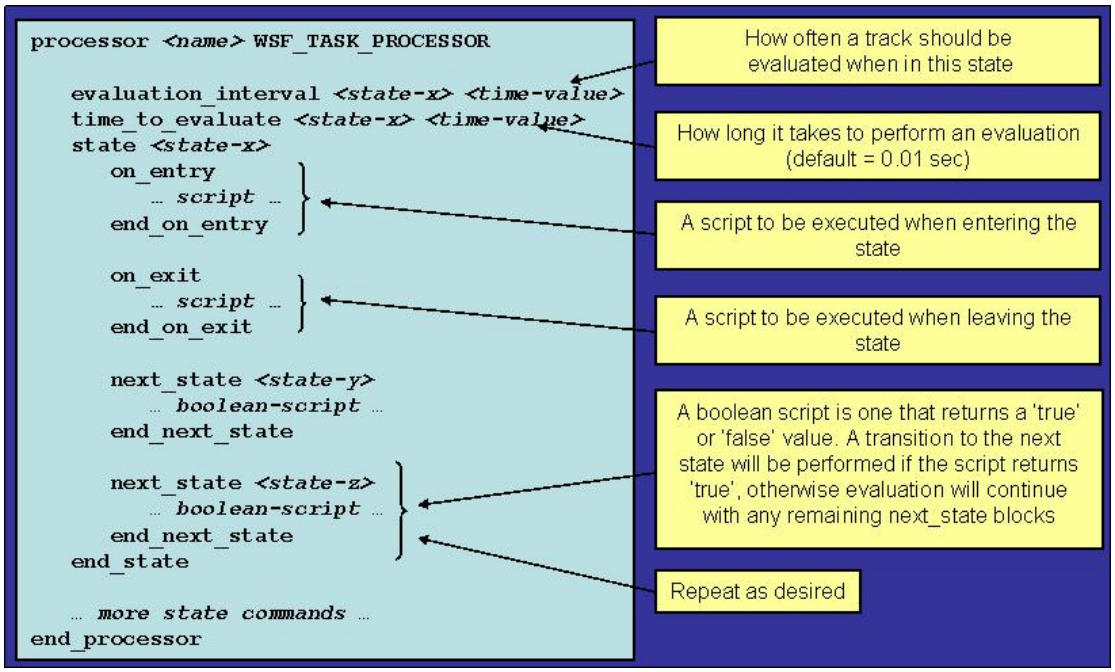
在以下示例中，显示了一个简单的状态图，其中有节点（标记为 A 到 D）和连接节点的弧线，表示转换规则。



状态“A”是起点。如果满足转换规则，它将转换到“B”。如果满足所有转换规则，它最终将转换到状态“C”，然后是“D”。在状态“B”和“C”时，如果某些转换规则不满足，它可以返回到“A”。可以用状态机来制作雷达系统的示例。状态“A”可以是搜索模式。状态“B”可以表示初始获取模式。状态“C”可以表示次级获取模式。状态“D”可以是跟踪模式。

状态机输入

以下图形显示了如何在 WSF 中的任务处理器内定义状态机。



任务处理器中可用的预定义变量

任务处理器可以访问一些预定义的变量，以简化脚本的创建，如下所示：

- TRACK 变量

  TRACK 变量是 WsfLocalTrack 类型，指的是当前正在评估的轨迹。由于 WsfLocalTrack 继承自 WsfTrack，因此 WsfTrack 中的所有方法也可用。
- PROCESSOR 变量

  PROCESSOR 变量（或已弃用的“this”）是 WsfTaskManager 类型，指的是 WSF_TASK_PROCESSOR。WsfTaskManager、WsfTaskProcessor、WsfProcessor、WsfPlatformPart 和 WsfObject 的所有方法都可以使用。
- PLATFORM 变量

  PLATFORM 变量是 WsfPlatform 类型，指的是包含处理器的平台。WsfPlatform 和 WsfObject 的所有方法都可以使用。
- MESSAGE 变量

  MESSAGE 变量是 WsfMessage 类型。WsfMessage 的所有方法都可以使用。
- TIME_NOW 变量

  TIME_NOW 是一个简单的“双精度”变量，表示自模拟开始以来的秒数。
- MATH 变量

  MATH 允许使用数学函数。参见 Math。

  有关其他组件中可用变量的信息，请参见：Common_Script_Interface。

状态机的评估过程

- 请求一个“思考者(thinker)”进程。
- 当一个思考者可用时，将其标记为忙碌状态，持续时间为当前状态的“time_to_evaluate”。
  - 模拟人或机器完成某事所需的时间。

    - 由正在评估的状态的“time_to_think”值控制。
    - 可以被重载。
- 如果思考者不可用（未忙于其他评估），评估将被放入“待处理队列”。
    - 将由下一个可用的思考者处理。
- WSF_TASK_PROCESSOR 可以配置多个思考者。
    - 命令：“number_of_servers <n>”（默认是 1）。
- 当思考者达到评估时间的末尾时，执行当前状态的规则：
    - 设置保留的脚本变量：TRACK、PLATFORM、PROCESSOR、MESSAGE、TIME_NOW。
    - 按出现顺序执行“next_state”脚本。
    - 如果没有返回 true，则保持在当前状态。
    - 将思考者标记为“非忙碌”。
    - 使用“evaluation_interval”安排下一个评估。
    - 如果一个返回 true，则转换到指定状态。
    - 执行当前状态的“on_exit”脚本。
    - 执行新状态的“on_entry”脚本。
    - 将“当前状态”设置为“新状态”。
    - 通过新状态的“time_to_evaluate”扩展思考者时间。
    - 等待思考者完成并评估规则。

任务的分配和执行

任务管理器可以向下属分配（或取消）任务。WsfTaskManager 中可用于分配或取消任务的一些方法包括：

- AssignTask（也可以使用 FireAt、StartTracking 和 StartJamming 用于武器）
- CancelTask
- TasksAssignedFor
- AssigneesForTask

任务分配/取消/状态消息通过通信网络传递（如果不是本地任务），并可能因距离或干扰而丢失。如果任务的受让人被消灭，所有分配给受让人的任务将自动在分配者处取消。

- 当收到任务分配时：
    - 使用消息中提供的轨迹更新轨迹管理器。
    - 如果无法与现有轨迹关联，则创建一个“本地轨迹”。
    - （分配者和受让人的轨迹 ID 不同！）
    - 增加轨迹的“锁定计数”。
    - 防止在任务活动期间清除本地轨迹。
    - 添加到接收到的任务列表中。
    - 将反映在 TasksReceivedFor 返回的值中。
    - 如果受让人之前不知道目标，将为目标创建并启动一个状态机。
    - 请求立即评估，等待思考者可用。
    - 不会延迟等待下一个评估周期。
- 当收到任务取消时：
    - 从接收到的任务列表中清除条目。
    - 不再反映在 TasksReceivedFor 返回的值中。
    - 向分配者发送确认。

- 调用“on_task_cancel”脚本（如果已定义）。
- 释放与任务相关的任何传感器或武器。
- （仅适用于 FireAt、StartTracking 和 StartJamming 任务。）
- 减少本地轨迹的锁定计数。
- 如果没有其他任务分配且轨迹超过清除间隔，则允许清除本地轨迹。
- 如果本地轨迹被清除：
  - 销毁与轨迹相关的状态机。
  - 调用“on_track_drop”脚本（如果已定义）。

1.2.4. 子系统几何考虑 Subsystem Geometry Considerations

概述

由于 WSF 试图表示以多种方式运行的子系统（传感器、武器或通信），因此它提供的定义属性（如几何限制）的机制可能相当复杂。本文档提供了一些关于这些机制如何运行的清晰说明，并指导如何定义将表现得像真实子系统的子系统模型。

从几何角度来看，子系统可以大致分为三类：

- 固定指向 - 指向角度始终朝向相同方向。
- 扫描 - 指向角度以规则模式移动。
- 命令指向 - 指向角度由命令选择（如跟踪）。

根据其操作模式，子系统可能表现出这三种特征。在某些情况下，命令指向和扫描可以一起使用（例如，指示获取或指示搜索）。在扫描或命令指向子系统中，改变指向角度的机制可能是机械的、电气的、光学的或通过其他方式。无论如何，指向角度的移动可能在方向上受到限制（例如，仅在方位上移动，仅在仰角上移动，或在任何方向上移动），并且在允许的方向上移动的量也可能受到限制。

坐标系统

最终，子系统与其他平台或子系统交互（例如，感知，发送消息）。当发生交互时，物体位于空间中的某个位置并具有一定的方向。子系统本身相对于其宿主平台具有位置和方向，并可能使用扫描“波束”。由于交互的最终成功或失败在很大程度上取决于几何，因此了解所使用的坐标系统及其关键参数的定义是基本的。

在交互过程中使用多达七个坐标系统：

- 世界坐标系统（WCS）
- 实体坐标系统（ECS）
- 部件坐标系统（PCS）
- 指示坐标系统（CCS）
- 扫描坐标系统（SCS）
- 波束坐标系统（BCS）
- 天线坐标系统（ACS）

常规武器，如导弹或枪支，通常仅使用前四个坐标系统（即 WCS、ECS、PCS 和 CCS）。传感器和通信设备还使用后三个（即 SCS、BCS 和 ACS）。注意，“波束”和“天线”这两个术语可能过于反映射频的性质。您可以将“波束”视为瞬时视角，将“天线”视为信号传输或接收的孔径。

以下部分定义了各种坐标系统的形成方式以及用于定义过程中使用的参数的命令。

世界坐标系统

世界坐标系统（WCS）是一个右手笛卡尔系统，定义如下：

- 原点位于地球中心。
- +X 轴通过 0N, 0E。
- +Y 轴通过 0N, 90E。
- +Z 轴通过 90N（北极）。
- 地球表面被建模为由 WGS-84 标准（NIMA TR-8350.2）定义的扁椭球体。

实体坐标系统

实体坐标系统（ECS）是一个刚性附着在平台（实体）上的坐标框架，是一个右手笛卡尔系统，定义如下：

- 原点位于实体中心。
- +X 轴从实体前部伸出。对于飞机，这将是从机头向前。
- +Y 轴从实体右侧伸出（当沿+X 轴向下看时）。对于飞机，这将是相对于飞行员的右翼。
- +Z 轴从实体底部伸出。
- 偏航是绕 Z 轴的旋转。正偏航向右。对于飞机，这将使机头相对于飞行员向右移动。
- 俯仰是绕 Y 轴的旋转。正俯仰抬高+X 轴。对于飞机，这将抬高机头。
- 滚转是绕 X 轴的旋转。正滚转降低+Y 轴。对于飞机，这将降低右翼。

部件坐标系统

部件坐标系统（PCS）是一个刚性附着在子系统上的坐标框架，仅是实体坐标系统（ECS）的平移和旋转。部件的位置和方向是相对于 ECS 定义的。相对位置和方向可以通过以下两种方式之一定义：

- 使用位置、偏航、俯仰和滚转静态定义。
- 使用 WsfArticulatedPart.SetYaw、WsfArticulatedPart.SetPitch 和 WsfArticulatedPart.SetRoll 脚本命令动态定义。

由 azimuth_slew_limits 和:command_.articulated_part.elevation_slew_limits 定义的回转限制是相对于此坐标系统定义的，表示系统可以实现的指向的绝对限制。传感器模式特定的指示限制覆盖（由 azimuth_cue_limits 和 elevation_cue_limits 定义）也相对于此坐标系统定义。

一个常见的错误是使用俯仰命令来定义绕其 Z 轴旋转的系统的天线倾斜角（例如，机场监视雷达、导弹发射器）。不幸的是，俯仰会倾斜整个坐标框架，包括 Z 轴！对于希望 Z 轴保持垂直的情况，应使用以下命令之一：

- 对于如 SAM 发射器或坦克炮塔的平台使用 tilt。
- 对于简单的单孔径（波束）系统或使用机体坐标进行扫描限制的电子扫描系统使用 antenna_tilt。
- 对于堆叠波束雷达系统使用 beam_tilt。

指示坐标系统

指示坐标系统（CCS）是应用指示命令后的 PCS。如果子系统具有非固定的:command_.articulated_part.slew_mode 或 cue_mode，则可以“指示”子系统。指示子系统

的命令包括：

- WsfArticulatedPart 中的任何 CueTo 命令。
- WsfTaskManager 中的 StartTracking 或 StartJamming 命令。

如果子系统可以被指示并且存在指示，则 CCS 的确定如下：

- 计算指示点相对于 PCS 的方位和仰角指示角。
- 确定“活动”方位和仰角指示模式和指示限制。
  - 通常由 slew_mode 和 elevation_slew_limits 定义。
  - 活动指示模式和指示限制可选地被传感器模式特定的 cue_mode、azimuth_cue_limits、elevation_cue_limits 覆盖。
- 确定相对于 PCS 的最终方位指示角。
  - 如果子系统可以在方位上指示（活动指示模式是方位或方位和仰角），则通过上一步定义的活动方位指示角限制限制方位指示角。
  - 如果子系统不能在方位上指示（活动指示模式是固定或仰角），则方位指示角为零。
- 确定相对于 PCS 的最终仰角指示角。
  - 如果子系统可以在仰角上指示（活动指示模式是仰角或方位和仰角），则通过上一步定义的活动仰角指示角限制限制仰角指示角。
  - 如果子系统不能在仰角上指示（活动指示模式是固定或方位），则仰角指示角为零。
- 通过旋转 PCS 变换以先前步骤确定的方位和仰角指示角计算 CCS 变换。

如果子系统不能被指示或未定义指示，则 CCS 仅为 PCS。

扫描坐标系统

扫描坐标系统（SCS）定义“扫描模式”的原点和方向。它与 CCS 相同，除非 scan_stabilization 不是 none。如果 scan_stabilization 不是 none，则 SCS 将重新定向以实现扫描稳定化的效果。

在 azimuth_scan_limits、elevation_scan_limits、azimuth_field_of_view 和 elevation_field_of_view 命令中指定的角度是相对于 SCS 的。

波束坐标系统

波束坐标系统（BCS）定义了“波束”的瞬时位置。BCS 的 X 轴与波束的中心对齐。对于非扫描系统（即，scan_mode 为固定），BCS、扫描坐标系统（SCS）和提示坐标系统（CCS）应当相同。

BCS 的形成如下：

1) 计算目标的方位和仰角相对于 SCS。
2) 确定波束的方位角相对于 SCS：
   - 如果波束可以在方位上扫描（即，scan_mode 为方位或方位和仰角），则波束方位角为目标方位角，限制在 azimuth_scan_limits 定义的范围内。
   - 如果波束不能在方位上扫描（即，scan_mode 为固定或仰角），则波束方位角为零。
3) 确定波束的仰角相对于 SCS：
   - 如果波束可以在仰角上扫描（即，scan_mode 为仰角或方位和仰角），则仰角为目标仰角，限制在 elevation_scan_limits 定义的范围内。
   - 如果波束不能在仰角上扫描（即，scan_mode 为固定或方位），则波束仰角为零。
4) 确保波束位置不超过子系统的回转限制。具体操作如下：
   - 将波束的方位和仰角从 SCS 转换回提示坐标系统。

- 如有必要，调整转换后的波束方位和仰角，以确保每个角度与其各自的当前提示角之和不超过 azimuth_slew_limits 和 elevation_slew_limits 定义的各自限制。

5) BCS 最终通过旋转 CCS 以先前步骤确定的转换和限制的波束方位和仰角形成。

目标相对于 BCS 的方位和仰角用于确定射频交互的天线增益。

天线坐标系统

天线坐标系统（ACS）定义了“天线”的方向。对于非电子控制的系统（electronic_beam_steering 为无），BCS 和 ACS 将相同。对于电子控制的系统，ACS 的 X 轴将垂直于阵列的表面。

BCS 的 X 轴与 ACS 的 X 轴之间的角度用于计算波束控制损失。有关更多信息，请参见 electronic_beam_steering、electronic_beam_steering_limit 和 electronic_beam_steering_loss_exponent。

传感器和通信处理的一般流程

本节试图从几何角度描述传感器或通信尝试的一般流程。它不讨论系统特定的处理。

1) 计算目标的距离并与模式特定的 minimum_range 和 maximum_range 进行比较。如果不在限制范围内，则抑制其余的检测处理。
2) 计算目标与传感器之间的相对高度，然后与模式特定的 minimum_altitude 和 maximum_altitude 进行比较。如果不在相对高度限制范围内，则抑制其余的检测处理。注意，传感器下方的高度限制应为负数。
3) 更新子系统的方向以反映任何潜在的提示（计算部件坐标系统和提示坐标系统）。
4) 计算目标相对于扫描坐标系统的方向并与 azimuth_field_of_view 和 elevation_field_of_view 进行比较。如果不在限制范围内，则抑制其余的检测处理。
5) 设置发射器/接收器波束位置（即，计算波束的波束坐标系统和天线坐标系统）。

在此之后，处理变得特定于传感器。例如，对于雷达，目标相对于发射器和接收器 BCS（以及可能的 ACS 对于电子控制系统）的方向将用于推导天线增益。

1.2.5. 子系统位置、方向、回转和扫描 Subsystem location, orientation, slewing and scanning

概述

以下子命令是通信、传感器和武器子系统定义中常见的。它们定义了对象相对于宿主平台的位置、方向、回转和扫描限制，以及视场。

定义标称子系统位置和方向

这些命令定义了子系统坐标框架（即，传感器可以回转或扫描的坐标框架）。

- location <x> <y> <z> <length-units>

指定子系统坐标框架的原点相对于平台原点的位置。这是在实体坐标系统中指定的，其中+X 为前方，+Y 为右侧（当向前看时），+Z 为下方。此值与偏航、俯仰和滚转的值一起定义了子系统坐标框架的位置和方向。

默认：0 0 0 米

- yaw <angle>

- pitch <angle>
- roll <angle>

指定子系统坐标框架的方向。

默认：全部 0.0 度

注意：这些值不应用于指定扫描系统的天线倾斜角。为此目的，请在接收器和发射器块中使用 antenna_tilt 子命令。

定义回转模式和限制

回转模式（及相应的限制）定义了子系统响应提示的能力。如果系统未被提示，则其方向由偏航、俯仰和滚转指定。

- slew_mode [fixed | azimuth | elevation | both]

指示子系统如何响应提示：

  - fixed 系统无法被提示。这是默认设置。
  - azimuth 系统只能在方位上被提示。
  - elevation 系统只能在仰角上被提示。
  - both 系统在方位和仰角上都可以被提示。

默认：fixed

- azimuth_slew_limits <min-angle-value> <max-angle-value>

指定子系统在方位上可以回转的最小和最大角度。这些值仅在 slew_mode 为 azimuth 或 both 时适用。限制是在子系统坐标框架中指定的。

默认：-180.0 度到 180 度

- elevation_slew_limits <min-angle-value> <max-angle-value>

指定子系统在仰角上可以回转的最小和最大角度。这些值仅在 slew_mode 为 elevation 或 both 时适用。限制是在系统坐标框架中指定的。

默认：-90.0 度到 90 度

定义扫描模式和限制

扫描是关于当前提示（或如果未定义提示，则关于标称子系统坐标框架）系统地搜索的过程。

- scan_mode [fixed | azimuth | elevation | both]

指示子系统如何相对于当前提示进行扫描：

  - fixed 子系统不移动。这是默认设置。
  - azimuth 子系统仅在方位上扫描。
  - elevation 子系统仅在仰角上扫描。
  - both 子系统在方位和仰角上都扫描。

默认：fixed

- azimuth_scan_limits <min-angle-value> <max-angle-value>

指定子系统在方位上可以扫描的最小和最大角度。这些值仅在 scan_mode 为 azimuth 或 both 时适用。限制是相对于当前提示的。

默认：-180.0 度到 180 度

- elevation_scan_limits <min-angle-value> <max-angle-value>

指定子系统在仰角上可以扫描的最小和最大角度。这些值仅在 scan_mode 为 elevation

或 both 时适用。限制是相对于当前提示的。

默认：-90.0 度到 90 度

定义视场

视场定义（方位、仰角和范围）是完全可选的。如果目标对象在视场之外，则可以绕过通信、干扰或感知机会的其余计算。

- azimuth_field_of_view <min-angle-value> <max-angle-value>

指定子系统在方位上可以看到的最小和最大角度。限制是相对于当前提示的。通常，这些值应大于或等于 azimuth_scan_limits（可能考虑到子系统定位到其扫描限制时波束的宽度）。

默认：-180.0 度到 180 度

注意：此值仅用于初步筛选以确定对象是否可以与另一个对象交互。

- elevation_field_of_view <min-angle-value> <max-angle-value>

指定子系统在仰角上可以看到的最小和最大角度。限制是相对于当前提示的。通常，这些值应大于或等于 azimuth_scan_limits（可能考虑到子系统定位到其扫描限制时波束的宽度）。

默认：-90.0 度到 90 度

注意：此值仅用于初步筛选以确定子系统是否可以与另一个对象交互。

- minimum_range <length-value>

指定系统可以与另一个对象交互的最小范围。

默认：0 米

注意：此值仅用于初步筛选以确定子系统是否可以与另一个对象交互。

- maximum_range <length-value>

指定子系统可以与另一个对象交互的最大范围。

默认：无限

注意：此值仅用于初步筛选以确定子系统是否可以与另一个对象交互。

示例

这些子命令的交互可能会令人困惑。本节将尝试提供一些提示和示例。

- 机械扫描的地面预警雷达

这种类型的系统通常只是旋转。它不响应指示，因此 slew_mode 为 fixed。

```
# z 坐标给出天线高度
location 0 0 -10 meters

#  天线从扫描平面向上倾斜 5 度
#  不要使用“pitch”命令，因为它会倾斜扫描平面！
antenna_tilt 5 degrees

slew_mode fixed
scan_mode azimuth
azimuth_scan_limits -180 degrees 180 degrees
```

- 高度测量系统

这种类型的系统通常被提示到特定的方位，然后上下“点头”。

```
slew_mode azimuth
azimuth_slew_limits -180 degrees 180 degrees
scan_mode elevation
elevation_scan_limits 0 deg 50 deg
```

- 获取系统

  这种类型的系统被提示到一个方位和仰角，然后搜索一个小体积。

```
slew_mode both
azimuth_slew_limits -180 degrees 180 degrees
elevation_slew_limits 0 degrees 40 degrees
scan_mode both
azimuth_scan_limits -2 degrees 2 degrees
elevation_scan_limits -2 degrees 2 degrees
```

- 跟踪系统

  这种类型的系统不断响应跟踪而被重新提示。

```
slew_mode both
azimuth_slew_limits -180 degrees 180 degrees
elevation_slew_limits 0 80 degrees
scan_mode fixed
```

1.2.6. 通信、传感器和干扰系统方程 Communication, Sensor and Jamming Systems Equations

Date: 25 February 2009

概述

本文档的目的是描述 WSF 中对象之间交互所使用的方程和算法。这包括：

- 传感器交互
- 通信交互
- 干扰（干扰）交互

通用射频方程

WSF 利用一组通用类来封装射频（RF）交互中涉及的组件（实际上，这些类的一些功能也用于非 RF 交互，但这在这里并不重要）。本文档的第一部分将处理信号传输和接收的基础知识。文档的后续部分将处理特定用途（雷达、SAR、ESM、干扰、通信）。

忽略接收信号处理的细节，RF 交互分为两类：

- 直接或单向：即发射信号直接到达接收器
- 间接或双向：即发射信号从物体反射然后被接收

接收信号功率的计算可以分为几个独立的步骤：

- 从发射天线发射
- 向目标或接收器传播
- 对于间接或双向交互：

- 从目标反射
- 从目标传播到接收器
- 由接收天线接收

直接发射功率的计算

$$P_x = P_{peak} \times \frac{G_x}{L_x} \text{(RF. 1)}$$

符号	来源	描述
G_x	transmitter antenna_pattern	发射天线在目标物体（接收器或平台）方向的增益。这包括任何电子波束控制损失（方程 RF.6）。
L_x	transmitter internal_loss	发射机中从电源到天线之间的内部损耗。
P_{peak}	transmitter power	发射机的峰值功率。这应该是单个脉冲的功率。
P_x	Computed	计算出的发射功率。

自由空间信号传播

从源（s）到目的地（d）的自由空间信号传播使用以下方程计算。在单向交互中，“s”和“d”分别是发射器和接收器（方程 RF.2b）。在双向交互中，有两个传播路径。第一个是从发射器到目标（方程 RF.2c），第二个是从目标到接收器（方程 RF.2d）。

$D_{sd} = P_s \times \frac{A_{sd}}{4\pi R_{sd}^2}$	一般形式 General form	(RF.2a)
$D_{xR} = P_x \times \frac{A_{xr}}{4\pi R_{xr}^2}$	发射器到接收器 Transmitter - to - receiver	(RF.2b)
$D_{xt} = P_x \times \frac{A_{xt}}{4\pi R_{xt}^2}$	发射器到目标 Transmitter - to - target	(RF.2c)
$D_{tr} = P_t \times \frac{A_{tr}}{4\pi R_{tr}^2}$	目标到接收器 Target - to - receiver	(RF.2d)

符号	来源	描述
A_{sd}	transmitter attenuation_model	计算信号从源（s）传播到目的地（d）时大气衰减影响后剩余的信号分数。
D_{sd}	Computed	从源（s）发出的在目的地（d）计算出的自由空间功率密度。
P_s	Computed	从源（s）发射的功率。这将是发射功率（方程 RF.1）或从目标反射的功率（方程 RF.3）。
R_{sd}	Computed	从源（s）到目的地（d）的斜距。

反射自由空间信号

反射自由空间信号的目标有效地创建了一个新的“发射源”。源的功率只是入射信号的信号密度与反射源的有效面积的乘积。反射器可以是一个平台（例如在执行双向雷达交互时）或地球表面（在执行杂波计算时）。然后可以通过应用方程 RF.2 将反射功率传播到接收器。

$$P_t = D_{xt} \times \sigma_t$$

符号	来源	描述
D_{xt}	Equation RF.2c	发射器（x）发出的信号在目标（t）处的功率密度。
P_t	Computed	由入射信号从目标反射产生的功率。
σ_t	radar_signature	目标的雷达截面。

接收自由空间信号

方程 RF.4a 用于直接、单向（通信、被动 RF 和干扰）。方程 RF.4b 用于双向（雷达、SAR）。

$P_r = D_{xr} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{POL}$	单向，发射器到接收器 One-way, Transmitter - to - receiver	(RF.4a)
$D_{xR} = P_x \times \frac{A_{xr}}{4\pi R_{xr}^2}$	双向，目标到接收器 Two-way, Target - to - receiver	(RF.2b)

符号	来源	描述
F_{BW}	See section 2.5	接收信号的分数，考虑到发射和接收器的频率/带宽可能不匹配。注意：这不适用于雷达交互，因为假设发射器和接收器是匹配的。
F_{POL}	transmitter_polarization receiver_polarization polarization_effects antenna_pattern	接收信号的分数，考虑到发射器和接收器的极化可能不匹配。注意：这不适用于雷达交互，因为假设发射器和接收器是匹配的。
F_{40}	transmitter propagation_model	模式传播因子。这考虑了直接和间接信号路径之间的建设性/破坏性干扰。注意：这目前仅对雷达交互实现。
D_{xr}	Equation RF.2b	发射器发出的信号在接收器处的功率密度。
D_{tr}	Equation RF.2d	从目标反射的信号在接收器处的功率密度。
G_r	receiver antenna_pattern	接收天线在目标物体（接收器或平台）方向的增益。这包括任何电子波束控制的影响（方程 RF.6）。
L_r	receiver internal_loss	接收器中从天线输出到接收器之间的内部损耗。
P_r	Computed	接收到的功率。

带宽比

因子F_{BW}用于考虑发射器的频谱可能与接收器的调谐带不匹配的事实。它是发射器频谱在接收器调谐带内的分数。

$F_{xl} = F_t - \frac{1}{2}B_x$	发射频谱的低频
$F_{xu} = F_t + \frac{1}{2}B_x$	发射频谱的高频
$F_{rl} = F_t - \frac{1}{2}B_r$	接收器的低调谐频率

$F_{ru} = F_t + \frac{1}{2}B_r$	接收器的高调谐频率

符号	来源	描述
B_r	receiver bandwidth	接收器的带宽。
B_t	transmitter bandwidth	发射器的带宽。
F_r	receiver frequency	接收器可以接收的频率范围的中心频率。
F_t	transmitter frequency	发射器频谱的中心频率。

F_{BW}的结果值取决于发射器和接收器的高低频率的关系。

$$F_{BW} = 0 \text{ 如果} F_{xu} \leq F_{rl}$$

$$F_{BW} = 0 \text{ 如果} F_{xl} \geq F_{ru}$$

$$F_{BW} = min\left(\frac{min(F_{xu}, F_{ru}) - max(F_{xl}, F_{rl})}{F_{xu} - F_{xl}}, 1.0\right) \text{ (RF.5)}$$

接收机噪声功率

以下定义适用于接收机噪声功率的计算：

符号	来源	描述
k	Internal constant	玻尔兹曼常数（1.3806505E-23 J/deg-K）
B	receiver bandwidth – or – transmitter pulse_width	接收机的带宽。如果未指定带宽且发射机是脉冲的，则带宽将计算为（1 / pulse_width）（即假设匹配滤波器）。
N	Computed	噪声功率
NF	receiver noise_figure	接收机噪声系数（默认 1.0）
T_0	Internal constant	标准温度（290 deg-K）
T_S	Computed	系统噪声温度。

噪声功率将使用以下过程计算。将使用第一个满足使用条件的步骤的值：

1) 如果指定了 noise_power，则使用定义的值。
2) 如果无法确定带宽，则使用-160 dBW 的值。
3) 如果指定了 noise_figure 并且省略了 antenna_ohmic_loss 和 receive_line_loss，则计算噪声功率为：

$$N = k \times T_0 \times B \times NF$$

4) 使用《雷达范围性能》，Lamont V. Blake，1986，Artech House, Inc.，第 4 章中定义的算法计算噪声功率。
   a. 天线噪声温度（T_{ant}天线指向角度的天空温度）：

$$T_a + \frac{(0.876 \times T_{ant} - 254.0)}{antenna_ohmic_loss} \quad \text{(RF.6b)}$$

   b. 接收线损耗引起的噪声温度贡献：

$$T_l = T_0 \times (receive_line_loss - 1.0) \text{ (RF.6c)}$$

   c. 接收机引起的噪声温度贡献：

$$T_r = T_0 \times (moise_figure - 1.0) \text{ (RF.6d)}$$

   d. 总系统温度：

$$T_s = T_a + T_l + (receive_line_loss \times T_r) \quad \text{(RF. 6e)}$$

e. 噪声功率：

$$N = k \times T_s \times B \quad \text{(RF. 6f)}$$

天线增益模式

每个发射机和接收机都有与之相关的天线增益模式。天线模式是使用全局 antenna_pattern 命令创建的。通过在发射机和接收机块中使用 antenna_pattern 命令将天线模式附加到发射机或接收机。如果未为发射机或接收机选择天线模式，则假设增益为 1.0。增益模式是方位角和仰角相对于模式原点（通常是瞄准线或指向角）的函数。对于给定的交互，计算相对于模式原点的兴趣点的方位角和仰角。

天线增益模式可以通过多种方式表示：

- 提供增益作为方位角和仰角函数的矩形表。
- ALARM 表。
- 均匀（恒定）模式。
- 圆形 sin(x)/x 模式。
- 矩形 sin(x)/x 模式。
- 余割模式。
- GENAP 模式（GENAP 是政府 TRAMS 模型中提供的广义天线模式例程功能的子集）。

可以使用表的集合来形成极化和频率的复合模式。

电子控制波束的增益可以选择性地修改，以包括将波束指向阵列法线以外角度的影响。此功能通过在发射机或接收机中使用 electronic_beam_steering 命令启用。使用以下方程：

$$G = G_0 \times cos^n(\theta) \quad \text{RF. 6}$$

符号	来源	描述
G_0	antenna_pattern	当观察兴趣点时天线的未修改增益。
G	Computed	包括电子波束控制影响的增益。
θ	Computed	天线面法线与兴趣点向量之间的角度。
N	electronic_beam_steering_loss_exponent	一个可选的指数，用于反映波束远离天线面法线时增益的退化量。

大气衰减

通过在发射机块中存在 atmospheric_attenuation 命令启用大气衰减的计算。

目前有两种可用的模型。这些模型是从 SUPPRESSOR 中提取的，目前仅适用于地面系统（表假设发射器在地面上）。

- blake - 由 L.V. Blake 编写的一个大气吸收模型，海军研究实验室。基于 42 条衰减曲线族，适用于 100 MHz 到 10 GHz 之间的频率和 0 到 10 度之间的仰角。曲线在 300 海里之外是平的。这些表发布在《雷达系统分析》，第 15.1 节，David K. Barton，Artech Publishing。
- earce - 来自 ESAMS/ALARM/RADGUNS 公共环境（EARCE）的一个大气吸收模型。这是一个预计算表的集合，适用于 100 MHz 到 18 GHz 和 27 GHz 到 40 GHz 之间的频率。频率低于 100 MHz 将假设为 100 MHz。18 GHz 到 27 GHz 之间和高于 40 GHz 的频率将使用一种非常计算密集的方法来确定衰减，应避免使用。

另一个基于国际电信联盟（ITU）建议 ITU-R P.676 的模型正在开发中。该实现将适用于空中和地面平台，并支持更广泛的频率范围。

传播算法

通过在发射机块中存在 propagation_model 命令启用传播效果（除了大气衰减）的计算。目前支持一个模型：

- fast_multipath - 在《雷达范围性能分析》，Lamont V. Blake，1986，Artech House, Inc. 中定义的方法的实现。它计算由于信号从圆形、粗糙地球反射而导致的建设性或破坏性干扰的效果。可以提供两个因素来定义反射点处表面的属性。

杂波算法

WSF 目前对表示杂波的能力非常有限。通过在接收机块中存在 clutter_model 命令启用杂波的使用。目前唯一的选项是使用杂波表，但尚未验证。

雷达传感器（WSF_RADAR_SENSOR）

WSF_RADAR_SENSOR 模型有效地计算单个脉冲（或连续波形）的功率，然后计算集成多个脉冲的效果。

接收功率计算

应用了方程 RF.1 到 RF.4 来计算单个脉冲（或连续波形）的接收功率。请注意，这不包括干扰处理，干扰是在单独的步骤中处理的。

$P_r = D_{tr} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{40}$ 从 RF.4b 得来

$= P_t \times \frac{A_{tr}}{4\pi R_{tr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{40}$ 从 RF.2d 得来

$= D_{xt} \times \sigma_t \times \frac{A_{tr}}{4\pi R_{tr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{40}$ 从 RF.3b 得来

$= P_x \times \frac{A_{xt}}{4\pi R_{xt}^2} \times \sigma_t \times \frac{A_{tr}}{4\pi R_{tr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{40}$ 从 RF.2c 得来

$= P_{peak} \times \frac{G_x}{L_x} \frac{A_{xt}}{4\pi R_{xt}^2} \times \sigma_t \times \frac{A_{tr}}{4\pi R_{tr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{40}$ 从 RF.1(Radar.1)得来

信号处理与检测

在信号处理和检测中，处理后的信号计算公式为：

$$S = P_r \times PCR \times G_i \times AF$$

符号	来源	描述
AF	adjustment_factor	一个通用的调整因子，用于考虑模型未提供的其他恒定效应。
G_i	integration_gain	由于多个脉冲的积分而产生的增益。如果指定了

		swerling_case，则此值在内部计算。
PCR	transmitter pulse_compression_ratio	脉冲压缩比。
P_r	Equation Radar.1	接收功率。
S	Computed	处理后的功率。

信噪比的计算公式为：

$$SN = \frac{S}{N + C + J} \quad \text{Radar. 3}$$

符号	来源	描述
C	receiver clutter_model	杂波功率。
J	Equation Jam.1	干扰机的入射功率。这是雷达接收器在检测交互时的入射功率之和。
N	Equation RF.6	接收器噪声功率。
S	Equation Radar.2	处理后的功率。
SN	Computed	信噪比（或干扰比）。

目标的检测由两种机制之一决定：

1) 简单的二元检测器：通过指定 detection_threshold 使用。如果信噪比超过此阈值，则宣布检测成功。
2) Marcum-Swerling 检测器：根据给定的信噪比产生检测概率。如果计算出的检测概率超过所需的检测概率，则宣布检测成功。选择此检测器需要使用 swerling_case、number_of_pulses_integrated、probability_of_false_alarm 和 detector_law 命令。

被动 RF 传感器 (WSF_ESM_SENSOR)

被动 RF 计算（如 ESM 和 RWR）使用单向方程。这里的“r”下标表示被动 RF 接收器，而“x”下标表示传感器、干扰机或通信发射机。扩展的方程如下：

$$P_r = D_{xr} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL} \quad \text{从 RF.4a 得来}$$

$$= P_x \times \frac{A_{xr}}{4\pi R_{xr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL} \quad \text{从 RF.2b 得来}$$

$$= P_{peak} \times \frac{G_x}{L_x} \times \frac{A_{xr}}{4\pi R_{xr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL} \quad \text{从 RF.1(ESM.1)得来}$$

信噪比的计算公式为：

$$SN = \frac{P_r}{N} \quad \text{ESM. 2}$$

符号	来源	描述
N	Equation RF.6	接收器噪声功率。
P_r	Equation ESM.1	处理后的功率。
SN	Computed	信噪比（或干扰比）。

成功检测的声明条件是信噪比超过以下定义的阈值：

- 如果传输信号是脉冲的，则使用 pulsed_detection_threshold。
- 如果传输信号是非脉冲的，则使用 continuous_detection_threshold。
- 如果未指定上述阈值，则使用 detection_threshold。

SAR 传感器 (WSF_SAR_SENSOR)

SAR 计算是雷达计算的扩展。

所需采集时间

用于计算所需采集时间以获得期望分辨率的方程为：

$$T_{OT} = \frac{KR_s\lambda}{2d_A V_G|sin(a)|cos(\delta)}$$

符号	来源	描述
d_A	Computed	所需的方位分辨率。
K	doppler_overcollect_ratio	过采集比（默认值为 1.0）。
R_S	Computed	从传感器到图像中心的斜距。
V_G	Computed	传感平台的地面速度。
λ	transmitter frequency -or- wavelength	发射信号的频率或波长。
α	Computed	传感平台的地面轨迹与图像中心向量之间的方位角。
δ	Computed	方位角。

射频干扰 (WSF_RF_JAMMER)

干扰计算使用单向方程，其中发射器是干扰机，接收器是雷达或通信接收器。干扰计算发生在雷达检测或通信尝试时。WSF 将汇总每个可能影响输出的干扰机的功率（即：如果有带内功率会影响接收器）。

“r”下标值用于传感器或通信接收器，“x”值用于干扰发射器。扩展方程如下：

$P_r = D_{xr} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL}$ 从 RF.4a 得来

$= P_x \times \frac{A_{xr}}{4\pi R_{xr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL}$ 从 RF.2b 得来

$= P_{peak} \times \frac{G_x}{L_x} \times \frac{A_{xr}}{4\pi R_{xr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL}$ 从 RF.1(Jam.1)得来

通信 (WSF_RADIO_TRANSCEIVER)

通信计算使用单向方程。

“r”下标值用于通信接收器，“x”值用于通信发射器。扩展方程如下：

$P_r = D_{xr} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL}$ 从 RF.4a 得来

$= P_x \times \frac{A_{xr}}{4\pi R_{xr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL}$ 从 RF.2b 得来

$= P_{peak} \times \frac{G_x}{L_x} \times \frac{A_{xr}}{4\pi R_{xr}^2} \times \frac{\lambda^2}{4\pi} \times \frac{G_r}{L_r} \times F_{BW} \times F_{FOL}$ 从 RF.1(Com.1)得来

信噪比计算为：

$$SN = \frac{P_r}{N + J}$$

符号	来源	描述
J	Equation Jam.1	表示入射干扰机功率。这是计算在交互时接收器上的入射功率之和。
N	Equation RF.6	表示接收器噪声功率。
P_r	Equation Comm.1	表示处理后的功率。
SN	Computed	计算得出的信噪比（或干扰比）。

如果信噪比 SN 超过接收器的检测阈值，则通信尝试将被宣布为成功。

红外传感器(WSF_IRST_SENSOR)

- 计算目标辐照度

确定背景辐射亮度。这包括一个相对简单的能力来考虑向天空或地面看的效果。

计算对比辐射强度：

$$I_c = I_s - L_{bkg} \times A_{proj}$$

符号	来源	描述
I_s	platform infrared_signature	表示目标的源辐射强度（红外辐射强度）。
L_{bkg}		背景辐射强度。
A_{proj}	platform optical_signature	表示传感器看到的目标投影面积。
I_c	Computed	计算得出的目标对比辐射强度。

计算大气透过率（信号沿路径传播后剩余的部分）：

计算有效目标辐照度（有时称为 CEI）：

$$E_{eff} = \frac{t \times I_c}{R^2}$$

- 调整安装效果

传感器通常安装在窗户后面，这会遮挡视野中的区域，或以其他方式减少信号。这种遮挡或信号减少统称为“安装效果”，通过在接收器块中使用 antenna_pattern 命令来考虑（虽然红外传感器中没有“天线”，但为了方便起见，将其视为天线）。该命令应参考天线增益模式，其中增益（或更可能是损耗）表示有效目标辐照度应如何调整以考虑安装效果，即：

$$E'_{eff} = E_{eff} \times G \text{ IRST.1}$$

其中 G 是感兴趣方向的“天线增益”。在窗户外的区域将增益设置为非常小的值，实际上使该区域的目标不可检测。

- 计算检测概率

  检测概率使用以下方程计算：

$$SN = \frac{E'_{eff}}{NEI}$$

$$\beta = SN - S_{thresh}$$

$$P_d = 1 - Q(\beta)$$

符号	来源	描述
E'_{eff}	Equation IRST.1	表示有效目标辐照度。
NEI	noise_equivalent_irradiance	表示传感器噪声的等效辐照度。
P_d	Computed	计算得出的检测概率。
$Q(\beta)$		高斯概率函数（参见《数学函数手册》，Abramowitz 和 Stegun，方程 26.2.5）。
SN	Computed	计算得出的信噪比。
S_{thresh}	detection_threshold	表示检测阈值。

1.2.7. 合成孔径雷达（SAR）图像形成方程 Synthetic Aperture Radar (SAR) Image Formation Equations

参考文献

1) 1.2.6 通信、传感器和干扰系统方程 Communication, Sensor and Jamming Systems Equations
2) 《合成孔径雷达性能限制》，第二版，Armin W. Doerry，桑迪亚国家实验室报告 SAND2006-0821。
3) 《合成孔径雷达模式约束》，2007 年 5 月 7 日，Matthew J. Renaud（波音公司）
4) 《雷达手册》，第二版，Merrill Skolnik

雷达距离方程

以下方程是 WSF 用于计算从单个射频信号脉冲接收到的功率的标准方程。该信号被发射，反射到物体上，然后被接收。此方程不假设发射器和接收器位于同一位置，也不考虑任何额外的信号处理技术，如脉冲压缩或多个脉冲的集成：

$$P_r = P_{peak} \frac{G_x}{L_x} \frac{A_{xt}}{4\pi R_{xt}^2} \sigma \frac{A_{tr}}{4\pi R_{tr}^2} \frac{\lambda^2}{4\pi} \frac{G_r}{L_r} F_{40} F_{BW} F_{POL}$$

符号	来源	描述
P_{peak}	transmitter power	发射功率，指的是峰值发射功率。
λ	transmitter frequency or wavelength	发射频率或波长，指辐射信号的波长。
G_x/G_r	transmitter / receiver	发射器/接收器天线增益，指发射器和接收器天线的

	antenna_pattern	增益。
L_x/L_r	transmitter / receiver internal_loss	发射器/接收器内部损耗，指发射器和接收器内部的损耗。
A_{xt}/A_{tr}	transmitter attenuation_model	发射器衰减模型，指由衰减模式计算的一次大气衰减因子（0..1）。
R_{xt}/R_{tr}	Computed	计算得出，指从发射器到目标和从目标到接收器的距离。
σ		目标的雷达截面（可能是目标或“分辨率单元”）。
F_{40}	transmitter propagation_model	发射器传播模型，指考虑直接和间接反射之间的干涉的传播因子。
F_{BW}		考虑发射信号带宽和接收器带宽不匹配的因子，主要用于雷达发射器与被动 RF 接收器或干扰器与雷达接收器之间的交互。
F_{POL}	transmitter / receiver polarization	发射器/接收器极化，指发射信号和接收天线的极化不匹配的因子。

对于 SAR，发射器和接收器是同一位置的，因此$R = R_{xt} = R_{tr}$且$A = A_{xt} = A_{tr}$。假设发射和接收天线的增益相同，即：$G = G_t = G_r$。此外，假设：

- 没有间接信号干扰主信号，因此$\mathrm{F}_{40} = 1$。
- 接收器的带宽已设定为捕获发射信号的全部带宽，因此$\mathrm{F}_{\mathrm{BW}} = 1$。
- 接收信号的极化与发射信号的极化相同，因此$\mathrm{F}_{\mathrm{POL}} = 1$。

如果我们还定义总大气衰减损耗为：

$$L_{atm} = \frac{1}{A_{xt}A_{tr}} = \frac{1}{A^2}$$

在上述假设下，我们得到发射器和接收器同一位置时从单个脉冲接收到的功率的熟悉方程：

$$P_r = P_{peak} \times \frac{G^2\lambda^2\sigma}{(4\pi)^3 R^4 L_x L_r L_{atm}}$$

这与参考文献 2 中的方程（1）相同，注意到：

$$L_{radar} = L_x L_r$$

并且方程（8）已用于替换$A_e = \frac{G_A\lambda^2}{4\pi}$

进一步注意到，参考文献 2 继续将有效孔径表示为实际孔径面积乘以“孔径效率”的乘积。我们在此不进行该步骤，假设任何孔径效率已在 WSF 天线模式中表示。

接收到的信号必须与系统内存在的噪声竞争。WSF 使用以下公式计算噪声功率：

符号	来源	描述
k	Internal constant	内部常数，玻尔兹曼常数（1.3806505E-23 J/deg-K）
B_N	receiver bandwidth	接收器带宽，指接收器的带宽。如果未指定带宽且指定了发射器脉冲宽度，则带宽将计算为（1 / 脉冲宽度）（即假设匹配滤波器）。
F_N	receiver noise_figure	接收器噪声系数，接收器噪声系数（默认 1.0）
N	Computed	计算得出，噪声功率。

T_0	Internal constant	内部常数，标准温度（290 deg-K）

天线端口的接收器噪声计算为：

$$N = kT_0B_NF_N$$

（注意：参考文献 1 的第 2.6 节描述了其他形式的噪声功率计算，但这些主要用于基于地面的系统。）

单个脉冲在天线端口的信噪比为：

$$SNR_{ant} = \frac{P_r}{N} = P_{peak}\frac{G^2\lambda^2\sigma}{(4\pi)^3R^4L_xL_rL_{atm}}\frac{1}{kT_0B_NF_N}$$

（这与参考文献 2 中的方程（5）相同，使用了上述替换。）

SAR 利用两种信号处理技术来增加图像中的有效信噪比。

- G_a：由于方位处理（相干脉冲集成）导致的 SNR 增益。
- G_r：由于距离处理（脉冲压缩）导致的 SMR 增益。

这将导致图像中目标的信噪比为：

$$SNR_{image} = SNR_{ant}G_aG_r = \frac{P_r}{N} = P_{peak}\frac{G^2\lambda^2\sigma}{(4\pi)^3R^4L_xL_rL_{atm}}\frac{1}{kT_0B_NF_N}G_aG_r$$

（这与参考文献 2 中的方程（11）相同，使用了上述替换。）

方位处理增益（相干脉冲积分）

合成孔径雷达（SAR）图像的生成涉及在某段适合产生所需质量图像的时间内相干地收集大量脉冲。

符号	来源	描述
f_p	transmitter pulse_repetition_frequency	脉冲重复频率
K_a	doppler_filter_broadening_factor	多普勒滤波器扩展因子
K_d	doppler_foldover_margin_factor	多普勒折叠裕度因子
t_D	dwell_time or computed	驻留时间或图像收集时间。
δ_a	resolution or computed	所需的方位分辨率。
V		载具速度向量。
θ_{sq}	computed	“斜视角”，定义为速度向量与图像区域中心的视线向量之间的角度。注意：在一些文档中，这将被测量为正侧角，导致使用 sin()和 cos()互换。
n_{image}	Computed	形成图像时收集的总脉冲数

引用 3 的方程（5）用于根据所需的横跨范围/方位分辨率计算驻留时间：

$$t_D = \frac{\lambda K_aR}{2V\delta_a sin(\theta_{sq})}$$

注意，WSF 允许用户指定所需的分辨率或驻留时间。在后一种情况下，WSF 将使用上述方程求解给定驻留时间的可实现分辨率。

方位增益是收集的总脉冲数，即收集时间乘以脉冲重复频率：

$$G_a = n_{image} = t_D f_p = \frac{\lambda K_a R f_p}{2V\delta_a sin(\theta_{sq})}$$

距离处理增益（脉冲压缩）

符号	来源	描述
τ_u	transmitter pulse_width	未压缩的脉冲宽度。
τ_u/τ_c	transmitter pulse_compression_ratio	脉冲压缩比。

由于脉冲压缩的距离处理增益是：

$$G_r = \frac{\tau_u}{\tau_c}$$

信噪比方程的各种形式

将G_a和G_r的结果代入SNR_{image}的方程：

$$SNR_{image} = P_{peak}\frac{G^2\lambda^2\sigma}{(4\pi)^3R^4L_xL_rL_{atm}}\frac{1}{kT_0B_NF_N}G_aG_r$$
$$= P_{peak}\frac{G^2\lambda^2\sigma}{(4\pi)^3R^4L_xL_rL_{atm}}\frac{1}{kT_0B_NF_N}\frac{\lambda K_a R f_p}{2V\delta_a sin(\theta_{sq})}\frac{\tau_u}{\tau_c}$$

这是 WSF 用于计算具有雷达截面积σ的物体回波的形式。这可以是目标或分辨率单元。文献中常见其它形式的方程。本节余下部分将展示上述方程如何等同。

在匹配滤波器的情况下：

$$B_N = \frac{1}{\tau_c}$$

代入：

$$SNR_{image} = P_{peak}\frac{G^2\lambda^2\sigma}{(4\pi)^3R^4L_xL_rL_{atm}}\frac{1}{kT_0B_NF_N}\frac{\lambda K_a R f_p}{2V\delta_a sin(\theta_{sq})}\frac{\tau_u}{\tau_c}$$
$$= P_{peak}\tau_u f_p\frac{G^2\lambda^3\sigma}{(4\pi)^3R^3L_xL_rL_{atm}}\frac{1}{kT_0F_N}\frac{K_a}{2V\delta_a sin(\theta_{sq})}$$
$$= P_{avg}\frac{G^2\lambda^3\sigma}{(4\pi)^3R^3L_xL_rL_{atm}}\frac{1}{kT_0F_N}\frac{K_a}{2V\delta_a sin(\theta_{sq})}$$

其中，平均功率定义为：

$$P_{avg} = P_{peak}\tau_u f_p$$

一种有趣的形式是当目标是一个裸分辨率单元（即：地面）。这有时被称为“杂波-噪声比”，或 CNR。参考 2 的方程（23）定义了分辨率单元的面积为：

$$\sigma = \sigma^0\delta_a\delta_{rg} = \sigma^0\delta_a\frac{\delta_r}{cos(\psi_g)}$$

符号	来源	描述
σ^0	backscatter_coefficient	后向散射系数
δ_r		距离分辨率（从有效脉冲宽度计算）
δ_{rg}		地面平面的距离分辨率

ψ_g		俯视角。视线向量与在观察点处表面切平面之间的角度。

代入：

$$SNR_{image} = P_{avg}\frac{G^2\lambda^3}{(4\pi)^3R^3L_xL_rL_{atm}}\sigma^0\delta_a\frac{\delta_r}{cos(\psi_g)}\frac{1}{kT_0F_N}\frac{K_a}{2V\delta_a sin(\theta_{sq})}$$
$$= P_{avg}\frac{G^2\lambda^3}{(4\pi)^3R^3L_xL_rL_{atm}}\frac{\sigma^0\delta_r}{cos(\psi_g)}\frac{1}{kT_0F_N}\frac{K_a}{2Vsin(\theta_{sq})}$$

基本上等同于参考 2 附录 B 中呈现的多种形式（然而，他们总是假设正侧收集，所以$sin(\theta_{sq})$总是 1）。

创建伪图像

WSF（Weapon System Framework）并不生成真实的图像，而是生成伪图像，这些伪图像显示图像中的物体、物体占据的分辨率单元（像素）数量以及物体的强度。

系统启动与目标列表构建：

用户将系统指向所需位置并启动系统。模型会构建可能出现在图像中的目标列表。目标列表将包括略微超出图像区域的目标，以考虑目标可能移动到图像中的情况。

数据采集与积累：

在周期性间隔（由‘frame_time’定义，默认 1 秒）内，模型计算并积累来自步骤 1 中每个目标的数据。这些检测结果将被积累，就像 SAR（合成孔径雷达）积累脉冲一样。如果目标在给定样本期间被地形遮挡，则在该间隔内不会有贡献脉冲。

伪图像生成：

在某个时刻，SAR 将被关闭。此时，模型将获取积累的结果并生成伪图像（WsfImage），并将包含图像的消息（WsfImageMessage）发送给已订阅的用户。

符号	来源	描述
t_F	frame_time	形成图像时样本之间的更新间隔。
n_{actual}		实际集成的脉冲数量。这可能与 n_image 不同，如果传感器在所需时间之前或之后关闭。
t_{sample}		样本的长度。
n_{sample}		在一个样本期间接收到的脉冲数量。
P_{sample}		在一个样本期间从特定目标接收到的信号。
NP_{sample}		在一个样本期间由特定目标覆盖的分辨率单元（像素）数量。
σ_{opt}	optical_signature	目标的光学特征。
P_{sum}		特定目标的采样接收信号的总和。
NP_{sum}		特定目标的采样像素计数的总和。
N_{seen}		特定目标可见的样本数量（未被地形遮挡）。
P_{ref}		对应于输出图像中零强度的参考信号。这通常是最小杂

		波-噪声比。
P_{range}		用于将接收到的信号缩放到[0..1]范围的归一化值。
CNR		来自单个分辨率单元的回波的预期信噪比。
CNR_{min}	detection_threshold	图像被声明为可接受的 CNR 的最小值。

计算步骤

1) 驻留时间与脉冲数量计算：
   - 计算预期的驻留时间（t_D）和要收集的脉冲数量（n_{sample}）。此外，还计算预期的 CNR 值。
2) 样本处理：
   - 对于步骤 2 中的每个样本，样本间隔期间接收到的脉冲数量为：$n_{sample} = f_p t_{sample}$。
   - 给定样本中目标占据的分辨率单元（像素）数量为目标的投影面积（光学截面）除以分辨率单元的大小：$NP_{sample} = \frac{\sigma_{opt}}{\delta_a \delta_r}$。
   - 样本间隔期间从目标接收到的每个分辨率单元的功率为：

$$P_{sample} = \frac{SNR_{image}}{N} \frac{n_{sample}}{n_{image}} \frac{1}{NP_{sample}}$$

3) 杂波-噪声比计算：
   - 计算实际收集的脉冲数量后，计算实现的杂波-噪声比：$CNR_{actual} = CNR \frac{n_{actual}}{n_{image}}$。
   - 如果CNR_{actual}大于或等于CNR_{min}，则图像将被声明为可接受，并将包含如下处理的目标。如果图像被声明为不可接受，则图像将不包含目标。
4) 目标信息生成：
   - 如果图像被声明为可接受，则为每个目标生成以下信息：
     - 目标占据的像素（分辨率单元）数量：$NP = \frac{NP_{sum}}{N_{seen}}$。
     - 像素的强度：$I = \frac{P_{sum} - P_{ref}}{P_{range}}$。小于零的值被限制为零，而大于一的值被限制为一。

1.2.8. 黑体辐射的普朗克定律在区间上的积分 Integration of Planck’s Law of Black Body Radiation Over an Interval

参考文献

1) 对黑体辐射普朗克定律的积分，W. K. Widger, Jr. 和 M. P. Woodall，《美国气象学会公报》，第 57 卷，第 10 期，1976 年 10 月，第 1217-1219 页。
2) 上述参考文献描述了一种在不使用需要大量迭代的数值方法的情况下，通过波长区间对黑体辐射的普朗克定律进行积分的机制。这里作为一个练习重复推导。

普朗克黑体辐射定律

普朗克黑体辐射定律表述为：

$$B(T,\lambda)=\frac{2hc^2}{\lambda^5}\frac{1}{e^{\frac{hc}{kT\lambda}}-1}$$

或者用辐射常数表示为：

$$B(T,\lambda)=\frac{c_{1L}}{\lambda^5}\frac{1}{e^{\frac{c_2}{T\lambda}}-1}\ (\text{单位：}\ \frac{W}{m^2\cdot sr\cdot m})$$

其中辐射常数定义为：

$\mathrm{c_{1L}}=2\mathrm{hc}^2$和$\mathrm{c_2}=\frac{\mathrm{hc}}{\mathrm{k}}$

这可以通过波长区间的积分来确定辐射亮度：

$$B_b(T)=\int_{\lambda_1}^{\lambda_2}B(T,\lambda)d\lambda\quad(\text{单位：}\ \frac{W}{m^2\cdot sr})$$

对于离散光谱带，这可以替换为：

$$B_b(T)=\int_{\lambda_1}^{\lambda_2}B(T,\lambda)d\lambda=\int_{\lambda_1}^{\infty}B(T,\lambda)d\lambda-\int_{\lambda_2}^{\infty}B(T,\lambda)d\lambda$$

假设变量替换：

$x=\frac{hc}{kT\lambda}$ 和 $\lambda=\frac{hc}{kTx}$

$dx=-\frac{hc}{kT\lambda^2}d\lambda$ 或 $d\lambda=-\frac{kT\lambda^2}{hc}dx=-\frac{kT}{hc}\frac{h^2c^2}{k^2T^2x^2}dx=-\frac{hc}{kTx^2}dx$

进行变量替换（暂时忽略积分上下限）：

$$\int\frac{2hc^2}{\lambda^5}\frac{1}{e^{\frac{hc}{kT\lambda}}-1}d\lambda=\int-2hc^2\frac{k^5T^5x^5}{h^5c^5}\frac{1}{e^x-1}\frac{hc}{kTx^2}dx$$

现在进行一些代数运算：

$$=\int-2hc^2\frac{k^5T^5x^5}{h^5c^5}\frac{hc}{kTx^2}\frac{1}{e^x-1}dx$$

$$=-2hc^2\frac{k^4T^4}{h^4c^4}\int x^3\frac{1}{e^x-1}dx$$

$$=-\frac{c_{1L}}{c_2^4}T^4\int x^3\frac{1}{e^x-1}dx$$

分子和分母同时乘以e^{-x}：

$$=-\frac{c_{1L}}{c_2^4}T^4\int x^3\frac{e^{-x}}{1-e^{-x}}dx$$

级数展开：

$$(1\pm z)^{-n}=1\mp nz+\frac{n(n+1)z^2}{2!}\mp\frac{n(n+1)(n+2)z^3}{3!}+\ldots(z^2<1)$$

如果令$z=e^{-x}$和$n=1$：

$$\frac{1}{1-e^{-x}}=(1-e^{-x})^{-1}=1+e^{-x}+e^{-2x}+e^{-3x}+\ldots$$

注意，x总是大于零，因为定义它的所有因子都是正的。因此，$0<z<1$ 和 $0<z^2<1$，如所需。积分可以重新写为：

$$=-\frac{c_{1L}}{c_2^4}T^4\int x^3e^{-x}(1+e^{-x}+e^{-2x}+e^{-3x}+..)\,dx=-\frac{c_{1L}}{c_2^4}T^4\sum_{n=1}^{\infty}\int x^3e^{-nx}\,dx$$

积分可以通过分部积分明确求解：

$$\int x^m e^{ax} dx = e^{ax} \sum_{r=0}^{m} (-1)^r \frac{m!\, x^{m-r}}{(m-r)!\, a^{r+1}}$$

对于$m = 3$ 和$a = -n$我们有：

$$\int x^3 e^{ax} dx = e^{-nx} \left[-\frac{3!\, x^3}{3!\,(-n)^1} + \frac{3!\, x^2}{2!\,(-n)^2} - \frac{3!\, x}{1!\,(-n)^3} + \frac{3!}{0!\,(-n)^4} \right]$$

$$= e^{-nx} \left[\frac{x^3}{n} + \frac{3x^2}{n^2} - \frac{6x}{n^3} + \frac{6}{n^4} \right]$$

将此代入求和中的积分：

$$= -\frac{c_{1L}}{c_2^4} T^4 \sum_{n=1}^{\infty} \int x^3 e^{-nx} dx = -\frac{c_{1L}}{c_2^4} T^4 \sum_{n=1}^{\infty} e^{-nx} \left[\frac{x^3}{n} + \frac{3x^2}{n^2} - \frac{6x}{n^3} + \frac{6}{n^4} \right]$$

论文中有一个额外的简化，被指出可以在相同准确度下减少迭代次数。我们不会使用它，因为它在低温（<100 K）和短波长（短于红外）时与直接数值积分有所不同，性能改进很小或没有观察到。但我会在此处包括它以供完整性。

注意以下级数：

$$ln(1+z) = z - \frac{z^2}{2} + \frac{z^3}{3} - \frac{z^4}{4} + \ldots \quad (-1 < z < 1)$$

令$z = -e^{-x}$，

$$ln(1-e^{-x}) = -e^{-x} - \frac{(e^{-x})^2}{2} + \frac{(e^{-x})^3}{3} - \frac{(e^{-x})^4}{4} + \ldots - \frac{(e^{-x})^n}{n}$$

求和的第一项可以替换为：

$$\sum_{n=1}^{\infty} e^{-nx} \frac{x^3}{n} = x^3 \sum_{n=1}^{\infty} \frac{(e^{-x})^n}{n} = -x^3 ln(1-e^{-x})$$

积分可以更新为：

$$= -\frac{c_{1L}}{c_2^4} T^4 [-x^3 ln(1-e^{-x})] \sum_{n=1}^{\infty} e^{-nx} \left[\frac{3x^2}{n^2} + \frac{6x}{n^3} + \frac{6}{n^4} \right]$$

$$= -\frac{c_{1L}}{c_2^4} T^4 [x^3 ln(1-e^{-x})] \sum_{n=1}^{\infty} e^{-nx} \left[\frac{3x^2}{n^2} + \frac{6x}{n^3} + \frac{6}{n^4} \right]$$

1.2.9. 电子战效果聚合 Electronic Warfare Effect Aggregation

参见：3.5.5.10.1 电子战效果聚合 Electronic Warfare Effect Aggregation

1.2.10. 坐标系统 Coordinate Systems

WSF 使用的坐标系统

WSF（战争模拟框架）使用的坐标系统是根据“IEEE Std. 1278.1-1995; 分布式模拟应用协议标准”定义的。

世界坐标系统 (WCS)

世界坐标系统 (WCS)，也称为地心地固坐标系 (ECEF)，是一个右手笛卡尔坐标系，定义如下：

- 原点位于地球中心。
- +X 轴通过 0° N, 0° E。
- +Y 轴通过 0° N, 90° E。

- +Z 轴通过 90° N（北极）。

地球表面被建模为一个由 WGS-84 标准（NIMA TR-8350.2）定义的扁球体。位置可以以纬度、经度和高度提供，但会转换为笛卡尔坐标。

地心惯性坐标系 (ECI)

地心惯性坐标系 (ECI) 与 WCS 类似，但相对于背景恒星固定，而不是地球。轴的方向参考 J2000 历元：2000 年 1 月 1 日，12:00 TT（地球时间），使得 x 和 y 轴定义了 J2000 的赤道平面。

- 原点位于地球中心。
- +X 轴指向春分点（太阳路径从南向北穿过赤道的点）。
- +Z 轴垂直于赤道平面，大致通过 90° N（北极）。
- +Y 轴垂直于 X 和 Z 轴，形成右手坐标系。

注意，AFSIM 的 ECI 坐标系统通常被称为地心天体参考系 (GCRF)。在忽略相对论修正的情况下，它是一个惯性坐标系。为了获得最高的 WCS-ECI 坐标转换精度，应定义极偏角、世界时差和原子时差。

真日期坐标系 (TOD)

真日期坐标系 (TOD) 类似于 ECI，但轴的方向参考感兴趣的日期（历元），使得 x 和 y 轴定义了该历元的赤道平面。

- 原点位于地球中心。
- +X 轴指向春分点。
- +Z 轴垂直于赤道平面（大致通过 90° N）。
- +Y 轴垂直于 X 和 Z 轴，形成右手坐标系。

注意，TOD 与 ECI 的区别在于 WCS-TOD 坐标转换中不考虑岁差和章动的影响。

实体坐标系统 (ECS)

实体坐标系统 (ECS) 是附加到实体（平台）的局部坐标系统。这是一个右手笛卡尔坐标系：

- 原点位于实体的中心。
- +X 轴从实体的前方伸出。
- +Y 轴从实体的右侧伸出（当沿+X 轴方向看时）。
- +Z 轴从实体的底部伸出。

偏航是围绕 Z 轴的旋转。正偏航向右。俯仰是围绕 Y 轴的旋转。正俯仰抬高+X 轴。滚转是围绕 X 轴的旋转。正滚转降低+Y 轴。

实体部件坐标系统 (PCS)

实体部件坐标系统 (PCS) 用于表示附加到实体（例如传感器）的部件的局部坐标系统。PCS 的原点位置和方向相对于附加部件的实体（平台）的 ECS 指定。PCS 的轴和角度约定与 ECS 相似：

- +X 轴从部件的前方伸出。
- +Y 轴从部件的右侧伸出（当沿+X 轴方向看时）。
- +Z 轴从部件的底部伸出。

偏航是围绕 Z 轴的旋转。正偏航向右。俯仰是围绕 Y 轴的旋转。正俯仰抬高+X 轴。滚转是围绕 X 轴的旋转。正滚转降低+Y 轴。

1.2.11. 通信入门 Communications Primer

引言

本入门手册旨在为用户提供有关通信功能的附加信息，包括现有的一般功能以及在 AFSIM 通信框架中扩展功能的机会。它并不是现有通信程序、术语、模型等现实世界等价物的替代品。一般来说，当 AFSIM 中的实现与传统理解有很大不同时，会特别指出以避免混淆。最后，本入门手册并不打算解释 AFSIM 以前版本的更改，但在解释通信功能时会提到这些差异（或“非差异”），以解释通信为何以这种方式运行。

地址分配

在 AFSIM 中，所有通信对象都需要有一个标识符或地址，以唯一标识模拟中存在的每个通信对象。最广泛认可的地址分配方法是 Internet 协议（IP）地址方案，它用于 AFSIM 中通信对象的内部地址分配。此外，AFSIM 使用 IPV4 无类别域间路由（CIDR）表示法进行通信的内部地址分配。选择这种地址分配方法是因为它的普遍性，以及它能够基于此类地址提供有关“组”或“网络”成员资格的附加信息，从而允许在通信系统中去除将网络显式建模为对象的需求。（有关更多详细信息，请参见网络）

IPV4 CIDR 作为 Internet 地址方案，显然不适用于所有通信对象，尤其是那些基于传统无线电通信的对象。然而，应该注意的是，尽管这些对象仍然需要基于 IPV4 CIDR 拥有一个地址以在 AFSIM 中保持一致的地址分配，但它们的行为并不受限于传统 IP 通信设备的预期方式。大多数情况下，用户可以选择忽略地址分配。

目前不支持 IPV6，但地址分配是基于未来可以实现其他地址方案（从基本地址类派生）的想法而开发的。一般来说，如果适当地限制 CIDR 值以避免过多的“稀疏”网络分配，IPV4 应该提供比模拟在任何给定时间能够处理的更多主机地址。

IPV4 CIDR 地址表示法使用一个 32 位值，排列为四个整数，以字节值的形式用句点（‘.’）分隔，范围从 0 到 255。AFSIM 中唯一受限的地址是 0.0.0.0，用于标识“空”地址（尚未在模拟中分配的地址）。现实世界中 IPV4 地址分配的复杂性（基于其他功能限制某些地址，主要限于 IP 功能，如多播、广播等）在 AFSIM 中未实现。使用 CIDR 表示法，地址附带一个附加值，指示用于地址网络本身的地址部分，这也直接指示在任何给定时间可以属于该网络的“主机”（单个通信对象）的数量。由于 CIDR 值是可变的，网络大小是灵活的。然而，应该注意的是，使用过大的网络大小而不实际使用提供的空间会限制模拟中可分配地址的总数。换句话说，如果手动确定地址分配，请限制小 CIDR 值的使用。CIDR 值限制在 1 到 31 之间，其含义如下所示：

CIDR	Subnet Mask	Number of Hosts (Per Network)
32	255.255.255.255	1
31	255.255.255.254	2
30	255.255.255.252	4
29	255.255.255.248	8
28	255.255.255.240	16
27	255.255.255.224	32
26	255.255.255.192	64
25	255.255.255.128	128
24	255.255.255.0	256
23	255.255.254.0	512
22	255.255.252.0	1024
21	255.255.248.0	2048
20	255.255.240.0	4096
19	255.255.224.0	8192
18	255.255.192.0	16384
17	255.255.128.0	32768
16	255.255.0.0	65536
15	255.254.0.0	131072
14	255.252.0.0	262144
13	255.248.0.0	524288
12	255.240.0.0	1048576
11	255.224.0.0	2097152
10	255.192.0.0	4194304
9	255.128.0.0	8388608
8	255.0.0.0	16777216
7	254.0.0.0	33554432
6	252.0.0.0	67108864
5	248.0.0.0	134217728
4	240.0.0.0	268435456
3	224.0.0.0	536870912
2	192.0.0.0	1073741824
1	128.0.0.0	2147483648
0	0.0.0.0	4294967296

- 提供通信对象所属网络的名称。在这种情况下，地址会自动分配给网络中最低的空闲地址。
- 提供 IPV4/CIDR 表示法中的显式地址。在这种情况下，通信对象将自动加入已存在的具有此地址的任何网络。
- 提供 IPV4/CIDR 表示法中的显式网络地址。如果没有与已存在网络的地址冲突，则会创建网络。
- 不做任何操作。通信会自动加入 AFSIM 中始终可用的通用网络，名为“default”。它被分配网络中最低可能的地址。

注意：AFSIM 中的“default”网络是保留地址 0.1.0.0/16。这提供了约 65,000 个可分配给该网络的通信。 警告：不要在 0.1.0.0 到 0.1.255.255 范围内创建网络，因为这些地址属于默认网络，如果尝试将导致模拟初始化错误。 警告：不要在与多播相关的范围 224.0.0.0

到 239.255.255.255 内创建网络。

网络

在 AFSIM 中，网络是通信对象的集合，通过其 IPV4 CIDR 地址的相似网络地址前缀分组。在 AFSIM 的早期版本中，通信仅限于同一网络内的对象，除非使用其他对象和设置（如现已弃用的 AFSIM_COMM_ROUTER）。这一概念严重限制了通信对象的灵活性和建模能力，因此被移除，转而将每个通信设备建模为可能能够与任何其他通信进行通信，限制由通信链路的存在和通信实现本身提供。为了实现任何两个通信对象之间的通信，只需在它们之间存在一个链路。

AFSIM 中的网络对象是用户可定义的对象，可以通过场景输入或脚本语言定义。这些对象对与网络相关的通信对象的成员资格、连接性和行为实施特定规则。此外，还提供了几种预定义的网络类型。未来可能会提供更多，或者用户可以扩展可用框架并添加自己的网络。AFSIM 中可用的网络对象通常实施网络拓扑（如环形、网状、星形等），但可以在网络定义中对添加/删除成员或添加/删除网络中的链路的操作实施任何规则。

未来计划还提供一种方法来驱动网络对象的更新，以便基于时间或事件的网络规则执行。这将提供一种基于协议使用、模拟环境或用户确定应驱动此类事件的一般事件来更新网络状态的方法。尽管目前不可用，但预计此机制将提供基于通信发现（和丢失）事件的自组织网络能力。

网络管理器

网络管理器是 AFSIM 中通信数据的主要收集点。模拟中的每个通信对象都需要向网络管理器注册以获取地址（由用户指定或动态分配）。每个通信都作为节点添加到图对象中，通信对象（节点）之间的通信能力表示为图中对象之间的边。网络管理器中维护的数据被视为“真实”数据，即 AFSIM 中任何给定时间点的实际通信状态。它还提供了一个对象，可以从 AFSIM 模拟对象中获取任何通信对象的地址、所属网络以及同一网络的其他成员列表。实际上，网络管理器充当 DNS 服务器。

路由和协议

更新至 AFSIM v2.5.0

在 AFSIM 中，路由是指确定从发送通信设备到目标接收者的通信路径的能力。这也意味着能够根据是否找到路径来判断消息是否可以发送到目的地。

路由在 AFSIM 中由路由器对象表示。这不应与作为网络硬件的路由器的通俗概念混淆，而是路由和路径查找能力的表示。它当然可以用于表示作为网络硬件的路由器，但这不是其主要目的。用户也不必对在无线电通信系统中使用路由器的概念感到惊讶，因为关键的无线电功能（如中继）是 AFSIM 中路由器对象所代表的一种路径查找形式，尽管是一种非常简单的形式。

路由器对象是平台部件，一个平台可以有任意数量的路由器与之关联。AFSIM 中始终提供一个路由器用于默认功能，以帮助保持向后兼容性，被称为默认路由器。每个通信（在路由器中称为接口）都与一个路由器关联，并且在任何给定时间只能与一个路由器关联。在简单的通信用例中，用户通常不需要了解这些概念，因为默认路由器存在于平台上，无需用户输入，平台上声明的所有通信都是默认路由器的接口。

路由器还包含可以模拟作为硬件的路由器概念的功能，例如通过在所有连接的成员之间创建通信链路来充当交换机（默认行为），如果需要，可以禁用。

路由器为消息找到路径的方式不是由路由器本身直接决定的，而是由其使用的路由协议（或多个协议）决定的。路由器支持任意数量的独特协议，在典型操作中，按优先顺序查询每个协议，直到其中一个协议确定到目的地的路径。如果找到这样的路径，它会指示通信对象将消息传输到路径中的下一个跳点。并非所有路由协议都需要为消息提供路径查找，有些协议启用不同的地址方案、路由知识更新，或以其他方式增强（或限制）路由器的功能。

每个路由器对象默认提供两个路由协议，尽管如果需要可以移除——WSF_COMM_ROUTER_PROTOCOL_LEGACY 和 WSF_COMM_ROUTER_PROTOCOL_MULTICAST。这些是通用协议实现，旨在提供 AFSIM 早期版本中固有的一些遗留功能，以便场景输入文件无需任何（或最少）更新即可在当前和未来版本的 AFSIM 中保持功能，特别是简单地将消息从通信 A 传递到通信 B。

最后，在增强通信框架的早期版本中，路由器提供了关于感知与真实网络状态使用的概念和用户切换。这已被移除，因为路由协议现在决定（逐案）它们是否使用模拟真实网络状态进行路径查找，或者它们是否在本地维护感知的网络状态，或两者的某种混合。由于路由器对象可以使用多个协议，这允许基于协议可用性在用户可配置的真实或感知路由选择中进行条件选择。

通信层实现和通信协议

AFSIM 中的通信对象使用 7 层 OSI 模型来组织通信类型的实现。基本上，每个通信对象包含一个称为协议栈的对象，其中包含多个“层”对象，用于处理从通信对象发送和接收的消息。当通信被指示发送消息时，该消息会通过栈中的每一层，每一层都有选择继续传递消息或中止过程的选项。接收消息的工作方式相同，只是消息在协议栈的层中以相反的方向（从下到上）遍历，具有相同的选项来将消息传递到更高层或中止接收。

请注意，尽管 AFSIM 通信对象使用此模型，但不要推断通信模型仅基于 IP。这只是支持通信对象建模的一种逻辑、灵活和可扩展的方式，无论它们如何运行或其预期用途，即使是基于电磁的。没有什么可以阻止最终用户仍然使用单层定义自己的通信对象，或者可能跳过层模型。这样提供只是为了减轻创建新模型、维护现有模型以及支持尚未创建的其他模型所产生的成本。

层结构的含义是每一层可以在通信内部和/或对消息执行操作，并以适合特定通信实现的方式处理消息。许多通信对象可能选择重用相同的层实现，或创建自己的实现，因为这些层可以通过 AFSIM 框架完全扩展。

目前，AFSIM 提供了以下具有基本功能的层：

- 应用层 - 在发送时，确定目标平台的可能接收者。
- 传输层 - 在发送时，将适当的传输协议附加到消息上，以便其他通信知道如何处理消息。
- 网络层
- 在发送时，使用路由器确定到目标接收者的路径，并选择“最佳”路径（由路由器的可用协议确定）。
- 在接收时，确定消息是否针对该平台，或者只是一个跳点。如果只是一个跳点，找到到目标的最佳路径，并转发消息。
- 数据链路层 - 在发送时，将消息放入队列，直到通信能够物理传输。
- 物理层 - 在发送时，确定消息传输涉及的时间，并发出电磁信号（如果适用）。通知

数据链路层（如果适用）何时可以发送更多排队的消息。如果使用可靠通信作为传输协议且发送失败，通知网络层发送失败（用于路由更新目的）。

代码库中还提供了其他层，但目前没有以有意义的方式使用。

可以向通信实现添加任意数量或类型的层，并可以根据需要进行修改。这允许创建更详细的实现，甚至在必要时可能实现数据包级别的实现。

通信还具有一个独特的组件，即通信协议。在 AFSIM 中，任何给定的通信实例可以关联任意数量的独特通信协议。通信协议修改通信模型层处理的默认行为，因为 AFSIM 提供的通信模型中的默认层实现会在典型的发送/接收操作中查询通信在每一层中遍历的通信协议。因此，通信协议允许修改现有通信模型而无需创建全新的模型来修改行为。这减少了创建新通信功能所需的开发量，允许它们在现有模型上使用，并大大减少了代码复制。目前，AFSIM 中仅预定义了一个通信协议 WSF_COMM_PROTOCOL_IGMP，它通过通信接口启用多播组成员资格，并允许通信对象接收多播消息。

指挥链、平台指定消息目标和通信的说明

以前版本的 AFSIM 使用指挥链来构建通信布局/网络状态表示。这一功能正随着 AFSIM 通信的更新逐步被移除。这样做的原因如下：

- 假设通信布局与指挥链相同是不现实的，即使这样做起来方便。
- 认为指挥链代表通信结构的假设，会导致行为上的假定，从而阻碍更高保真度的建模，并限制 AFSIM 作为工具进行实验的能力，尤其当通信是重要变量时。
- 这种模型曾在一些旧的仿真框架中使用，并在 AFSIM 中初步复制，以帮助行为的一致性验证和确认。如今，它限制了 AFSIM 的能力，尤其是在有效通信建模日益重要的情况下。
- 指挥链指定接收目标平台，而不是目标通信（见下文，了解为什么这有害）。

此外，在 AFSIM 中，指定通信消息的目标平台的功能正逐渐被移除，原因如下：

- 一个平台可以维护多个通信。简单地将平台指定为消息接收者并不指明哪个通信将接收消息。
- 因为上述原因，选择哪个通信接收消息属于假设行为，或者更糟糕的是，随机排序。这是不可接受的，因为模拟需要确定性和可重复性。
- 不知道消息接收的具体通信接口会导致难以调试场景建模中的错误，可能导致平台上的链接无法将传入消息路由到其正确的部分目的地，或者：
- 用户被迫在所有通信接口上始终如一地复制内部链接，以确保消息的正确内部平台路由，这就强制使用唯一的消息类型进行过滤，并增加了布局的复杂性。

分布式仿真与通信

在分布式环境中使用通信时，用户需要注意一些细节以确保功能正常。

- 默认情况下，通信设备可以在本地仿真环境中动态寻址。虽然这实际发生的方式超出了本文档的范围，但需要注意的是，当用户让仿真为通信设备分配地址而不明确定义时，生成的地址取决于网络中任何后续寻址的可用性，通信设备属于先到先得的基础上，从最低可用地址开始递增到最高。
- 这在分布式仿真用例中是问题，因为通信设备的排序可能在一个仿真实例与另一个仿真实例之间不一致。这可能导致地址分配冲突，显而易见的是，由于消息在其他仿真实例

中以发送仿真实例中的正确地址到达，但在接收的实例中错误，导致错误的交付和/或消息丢失。

- 目前，没有机制确保在使用动态寻址时跨分布式仿真实例进行正确和唯一的地址分配。强烈建议在分布式用例中使用通信的用户为其通信设备使用静态寻址。静态寻址规避了上述问题。此外，只要确保使用动态寻址的任何通信设备属于仅在同一仿真环境中定义和使用的网络，动态寻址在分布式用例中仍然可行。因此，用户可以配置其场景以继续利用动态寻址，只要他们修改其网络布局以仅包含本地定义和管理的通信设备。

其他注意事项

在 AFSIM 中通信还有一些其他重要注意事项：

- 通信框架不传递端到端消息（以前的 AFSIM 版本在某些情况下会这样做）。发送消息可能会生成多跳路径。AFSIM 中的每个通信对象接收、处理并重新发送这些消息，直到到达目的地。
- 默认情况下，AFSIM 继续将对象分配到一个名为“default”的网状网络，并提供初始链接。
- 基于平台的消息发送在所有情况下都被移除。这包括任何基于指挥链的操作。新框架的保真度已足够高，不能再将消息概括为发送到一个平台引用，因为接收者可能不止一个通信。所有通信发送调用都需要一个发送通信和一个接收通信，以多种格式之一进行。目前，针对发送到平台的现有方法，所有可用的发送通信将发送消息，以提供一种临时的输入文件过渡方法。
- 网关仍然可用，任何失败的消息路由将自动将消息传递到定义的网关（假设用户选择的协议支持这一点）。网关是针对每个通信对象定义的，因此任何通信都可以定义自己的网关。使用旧的“default_gateway”命令不再支持。

1.2.12. 链接更新指南 Link Update Guide

概述

本指南旨在简要总结 AFSIM 2.4.0 中对 external_link 命令所做的更新，并帮助用户将现有的场景输入文件转换为新的输入格式。

一般说明

external_link 命令已更改，以符合 AFSIM 中新通信框架的要求。以前，只需指定通信的目标平台即可。新通信框架的保真度要求必须指定一个特定的通信对象作为任何通信的接收者。由于平台可能包含多个与之关联的通信对象，因此在 AFSIM 中进行任何通信特定任务时，仅提供平台信息已不再足够。因此，此更新是必要的，以确保在仿真运行时选择通信发送者或接收者时不会做出无效假设。

与 AFSIM 中的所有通信一样，外部链接基于消息的发送者和接收者之间存在通信链接的概念。如果不存在这样的链接（或链接系统），此命令将失败。

关于现有输入的警告

请注意，某些格式的 external_link 命令在此更新后仍然有效。例如，提供诸如 report_to commander via datalink 的命令在此更新后仍然是有效的命令。然而，这些命令的功能并不相

同！以前的版本会在需要时从“datalink”通信向指挥官平台发送消息，发送给第一个具有通信路径的接收者。在新版本中，除了仅传输给与发送者通信同名的接收者（在此例中为“datalink”）外，其他都相同。如果在接收者上找不到这样的通信，则不会发送消息。因此，2.4.0 之前和之后此命令的行为非常不同，即使语法完全相同。在使用新的 external_link 命令更新场景时，请特别注意这一点。

旧版指挥链的使用

在以前的版本中，分析师通常使用一个网络向上发送指挥链中的消息，另一个网络向下发送指挥链中的消息。通常，这些网络被称为类似“cmdr_net”和“sub_net”的名称。使用新的通信框架，这种结构不是必要的，但仍然可以使用。如果使用，请注意，external_link 命令始终从发送者的角度出发，因此任何发送给指挥官的命令应通过 cmdr_net 发送，但将由接收者在 sub_net 上接收。例如，report_to commander via cmdr_net to sub_net。在向下发送时则相反，例如，report_to subordinates via sub_net to cmdr_net 是适当的输入。在同级之间的消息在任一网络上都是有效的，如果它们都是相同平台类型，则不需要“to”命令，例如，report_to peers via sub_net，因为相同类型的平台通常在它们之间具有相同名称的通信。

旧版 WsfGroup 的使用

组可以继续用作外部链接的目标。然而，external_link 命令现在仅使用组中的通信对象作为目标，而不是平台。平台和 WsfGroup 中任何不是通信对象的其他对象将被忽略。

地址使用

虽然外部链接现在可以使用地址作为目标，但这不是一个选项，除非用户为特定目标或网络提供特定地址。没有静态地址分配的对象会被动态分配一个地址。仿真输入中的非常细微的变化可能会改变此地址分配，因此不要尝试在一次仿真运行中找到对象的地址，并在任何后续运行中修改输入以使用该值。对于任何两次仿真运行之间的动态地址，不提供任何保证。

1.2.13. 脚本 script

概述

Scripts 为用户提供了一种基于仿真中发生的事件执行复杂指令集的方法。其语言类似于 C#和 Java，对于具备基本编程技能的人来说应该很熟悉。它是块结构化的，包含熟悉的声明、赋值和流程控制语句，允许用户检查和操作仿真环境。

Scripts 本质上是由脚本编译器生成的指令列表，该编译器理解脚本语言语法并将其翻译为适当的指令。一旦编译，Scripts 可以在脚本上下文（执行上下文）中执行，该上下文负责解释脚本的指令并提供脚本与应用程序（即仿真）层之间的接口。在 WSF 中，执行上下文被链接形成树结构，允许子上下文继承其父上下文定义的 Scripts。

基本类型

Script 有四种基本类型。基本类型在赋值给变量和传递给函数时被复制。基本类型包括：

```
int：32 位整数，例如：int prime5 = 11;
bool：布尔值，‘true’或‘false’，例如：bool isTrue = true;
double：双精度浮点值，例如：double gravity = 9.8;
string：字符字符串，例如：string text = "Hello World";
```

基本类型支持多种运算符。‘int’和‘double’都支持基本算术和比较运算：

```
// 算术运算（+, -, *, / )
double abc = (5 + 2.5) / 2.0 - 1.0;
// 算术赋值: ( +=, -=, *=, /= )
abc += 5.0;    // abc 加 5
// 比较运算（>, >=, <, <=, ==, != )
bool isPositive = abc > 0.0;
```

‘string’支持比较运算符和‘+’作为连接：

```
string alphabet = "alphabet";
string zoo = "zoo";
bool trueVal = alphabet < zoo;
string combine = alphabet + " " + zoo;
```

‘bool’支持比较运算符：

```
bool trueVal = true != false;
```

Script 类

所有其他类型称为Script类。这些类型通过引用复制。可用的类型很多，详见Script Types：

```
// 定义一个非基本类型的变量。
// 在这个例子中，val 没有值，是'null'。
Vec3 val;

// 创建一个新的 Vec3 值
Vec3 val2 = Vec3();

// 调用返回值的静态方法的示例。
Vec3 val3 = Vec3.Construct(1,2,3);
```

方法

方法是存在于 Script 类上的函数。方法可以是静态的或非静态的。静态方法可以在没有脚本对象的情况下调用，例如：

```
Vec3.Construct(1,2,3);
```

非静态方法必须在有效对象上调用，例如 Magnitude()：

```
Vec3 v = Vec3.Construct(1,2,3);
double length = v.Magnitude();
```

在无效对象上调用非静态方法会导致运行时错误：

```
Vec3 v;
v.Magnitude(); // 产生错误，v 未初始化，因此为'null'
```

可以这样测试对象的有效性：

```
if (v) {
    double length = v.Magnitude();
}
```

Scripts

用户定义的函数称为‘Scripts’。通常 Scripts 具有以下语法：

```
script <type> <script-name>([<variable-declaration-list>])
    <script-commands...>
end_script
```

Scripts 可以调用在相同上下文中定义的其他 Scripts 或在父上下文中定义的 Scripts。例如：

```
script bool global_script()
    writeln("Global script called");
    return true;
end_script

platform plat WSF_PLATFORM
    script void local_script()
        writeln("Local script");
    end_script

    // on_initialize 是不同脚本语法的示例。
    on_initialize
        // 调用全局脚本，任何地方都可以访问
        global_script();
        // 调用本地脚本，仅在此平台上可访问
        local_script();

        // 要调用稍后定义的脚本，必须使用'extern'声明它
        extern bool global_later(double);

        global_later(123.0);

    end_on_initialize
end_platform
```

```
script bool global_later(double value)
    writeln("global_later(", value, ")");
end_script

platform plat2 WSF_PLATFORM
    // 此平台无法直接访问'plat'上的 Scripts
    //
    script void call_plat1()
        // 找到上面定义的平台
        WsfPlatform plat = WsfSimulation.FindPlatform("plat");

        // 这里，'.'操作符不能用于调用'plat'上的 local_script()
        // '->'操作符允许访问其他对象，但这种访问在启动时不检查。
        plat->local_script();

        // 如果出错，会在运行时发出错误。
        // 这会导致运行时错误：
        // plat->bad_script();
    end_script
end_platform
```

全局变量

全局变量允许从多个 Scripts 中存储和访问值。当定义全局变量时，它仅对其父上下文全局。例如，在平台上定义全局变量仅允许从该特定平台访问该变量。有多种定义全局变量的方法：

```
// 在平台外的'script_variables'块中定义的变量始终是全局的。
// 'x'是真正的'全局'变量，任何地方都可以使用
script_variables
    double x = 1.0;
end_script_variables

platform plat WSF_PLATFORM
    // y 是仅对位于'plat'上的 Scripts 可用的全局变量
    script_variables
        double y = x;
    end_script_variables

    script void test()
        // 全局变量可以像常规变量一样使用
        y += x;

        // 全局变量也可以在任何脚本中使用'global'关键字定义：
```

```
        global double z = y;
    end_script

end_platform

script void test2()
    // 找到上面定义的平台
    WsfPlatform plat = WsfSimulation.FindPlatform("plat");
    // '->'操作符可用于访问属于对象的变量。
    plat->y -= x;
end_script
```

静态变量

静态变量是只有一个实例并且只初始化一次的变量：

```
script void test_static()
    // 在这个例子中，x 在第一次调用此脚本时初始化为 1.0。
    // x 的值在对 test_static()的调用之间保持不变。
    // 这将输出 1234...每次调用 test_static()时输出一个数字。
    static double x = 1.0;
    write(x);
    x += 1;
end_script
```

类型转换

Casting 是将一个值从一种类型转换为另一种类型。转换的语法为(<type>)value。基本类型之间可以自由转换：

```
int five = (int) 5.5;
string fivePointFive = (string)5.5;
```

在非基本类型之间进行转换是允许的，但用户应谨慎操作：

```
WsfMessage msg = GetControlMessage();
((WsfControlMessage)msg).SetResource("my_resource");
```

运算符

- .：用于调用脚本类对象上的方法，或脚本类上的静态方法。
- ->：用于调用对象上的用户定义脚本，以及获取/设置对象上的用户定义脚本变量。
- +：用于数值相加和字符串连接。
- -：用于数值相减，例如：1 - 1。
- *：用于数值相乘，例如：2 * 2。
- /：用于数值相除，例如：4 / 2。
- >：用于大于比较，例如：1 > 0。

- >=：用于大于或等于比较，例如：1 >= 1。
- <：用于小于比较，例如：0 < 1。
- <=：用于小于或等于比较，例如：0 <= 1。
- ==：用于等于比较，例如：1 == 1。
- !=：用于不等于比较，例如：1 != 0。
- !：布尔非运算，例如：true == !false。
- ()：用于表达式的优先级，例如：(1+1)*2 == 4。
- (<type>)：类型转换运算符。
- =：赋值运算符，例如：double x = 2.0;。

详细信息

这是脚本语法的详细文档。

- 符号说明：
  - 尖括号（< >）用于标记类别。
  - 方括号（[ ]）用于标记可选项。
  - 花括号（{ }）用于标记重复项。
  - 单引号（‘ ‘）用于标记字面量。
  - 粗体文本表示保留字。

注意：完整的语言语法可以在 Scripting Language Grammar 中找到。

命令模板

脚本使用以下序列定义：

```
script <type> <script-name>([<variable-declaration-list>])
    <script-commands>
end_script
```

这种结构允许用户定义和执行复杂的指令集，利用脚本语言的语法和功能来操控仿真环境。

语言描述

该语言由以下构造组成：

<identifier> 标识符

标识符表示变量或函数的名称。标识符以字母（大写或小写）开头，后跟零个或多个字母、数字（0-9）或下划线（‘_’）。标识符是区分大小写的。因此，标识符‘x1’和‘X1’表示不同的变量。有效标识符的示例：

- i
- X1
- aLongIdentifier
- x_2

<type> 类型

每个变量都有一个‘类型’，定义了它可以包含的数据类型。数据主要有两种类型：<basic_type> 和 <complex_type>。所有类型都派生自一个称为 Object 的‘基’类型。

Object：所有其他类型都兼容的‘基’类型。

例如：

```
Object myObject;
myObject = 'hello';
myObject = 19;
```

<basic_type> 基本类型

脚本语言的类型与大多数现代编程语言提供的类型相匹配：

- int
- double
- bool
- string

<complex_type> 复杂类型

这些是更复杂的类型，通常由几个基本类型或其他复杂类型组成，通常包括可以访问和操作类型内数据的函数。复杂类型不能在脚本中定义；它们是在 C++中定义并导出以供脚本使用。复杂类型的完整列表可在 Script Types 部分找到。

<storage-type> 存储类型

当变量被声明时（见下文），内存存储类型要么是隐式的，要么是显式设置的。默认情况下（如果未指定存储类型），变量被认为是自动的，意味着它们是在当前<block>的内存空间中创建的。这也意味着它们仅在当前<block>及其内部嵌套的<block>中可用。除了自动变量，还有全局变量和静态变量。全局变量在全局内存中分配，并在所有脚本中可用。静态变量的工作方式与自动变量相同，只是它们的内存（及其当前值）在对给定脚本的调用之间被保留。

<expression> 表达式

表达式是任何导致单个值的内容。

例如：

```
10 * 3
('platform-1' == platform.Name()) && (5 < mX)
Foo()
mX
MATH.Pi()
(9.99 >= 1.0)
1.23
```

<expression-list> 表达式列表

以逗号分隔的<expression>列表。

<cast> 类型转换

表达式可以使用类型转换操作转换为另一种类型。在某些情况下这是必要的（参见 Script Types 部分中的 Iterator、ArrayIterator 和 MapIterator）。

```
'(' <type> ')' <expression>
```